```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("insurance_dataset.csv")

print("First 5 rows:")
display(df.head())

print("Dataset shape:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)
```

```
First 5 rows:
```

	age	gender	bmi	children	smoker	region	medical_history	family_medical_history	exercise_frequency	occupation	coverage_level	charges
0	46	male	21.45	5	yes	southeast	Diabetes	NaN	Never	Blue collar	Premium	20460.307669
1	25	female	25.38	2	yes	northwest	Diabetes	High blood pressure	Occasionally	White collar	Premium	20390.899218
2	38	male	44.88	2	yes	southwest	NaN	High blood pressure	Occasionally	Blue collar	Premium	20204.476302
3	25	male	19.89	0	no	northwest	NaN	Diabetes	Rarely	White collar	Standard	11789.029843
4	49	male	38.21	3	yes	northwest	Diabetes	High blood pressure	Rarely	White collar	Standard	19268.309838

```
Dataset shape: (154292, 12)

Column names:
['age', 'gender', 'bmi', 'children', 'smoker', 'region', 'medical_history', 'family_medical_history', 'exercise_frequency', 'occupation', 'coverage_level', 'charges']

Data types:
age                        int64
gender                    object
bmi                      float64
children                   int64
smoker                    object
region                    object
medical_history           object
family_medical_history    object
exercise_frequency        object
occupation                object
coverage_level            object
charges                  float64
dtype: object
```

```
print("Missing values in each column:")
print(df.isnull().sum())

print("\nTotal missing values:")
print(df.isnull().sum().sum())
```

```
Missing values in each column:
age                           0
gender                        0
bmi                           0
children                      0
smoker                        0
region                        1
medical_history           38942
family_medical_history    38665
exercise_frequency            1
occupation                    1
coverage_level                1
charges                       1
dtype: int64

Total missing values:
77612
```

```
print("Duplicate rows:", df.duplicated().sum())

df = df.drop_duplicates().copy()

print("Dataset shape after removing duplicates:", df.shape)
```

```
Duplicate rows: 0
Dataset shape after removing duplicates: (154292, 12)
```

```
# Fill missing categorical values
categorical_columns = df.select_dtypes(
    include=["object", "category"]
).columns

for col in categorical_columns:
    df[col] = df[col].fillna("Unknown")

# Fill missing numerical values with the median
numerical_columns = df.select_dtypes(
    include=np.number
).columns

for col in numerical_columns:
    if df[col].notna().any():
        df[col] = df[col].fillna(df[col].median())
```

```
print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
age                       0
gender                    0
bmi                       0
children                  0
smoker                    0
region                    0
medical_history           0
family_medical_history    0
exercise_frequency        0
occupation                0
coverage_level            0
charges                   0
dtype: int64
```

```
# Identify numerical and categorical columns
numerical_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    include=["object", "category"]
).columns.tolist()

print("Numerical columns:")
print(numerical_columns)

print("\nCategorical columns:")
print(categorical_columns)

# Convert categorical columns into numerical columns
df_encoded = pd.get_dummies(
    df,
    columns=categorical_columns,
    dtype=int
)

print("\nPreprocessed dataset:")
display(df_encoded.head())

print("Encoded dataset shape:", df_encoded.shape)
```

```
Numerical columns:
['age', 'bmi', 'children', 'charges']

Categorical columns:
['gender', 'smoker', 'region', 'medical_history', 'family_medical_history', 'exercise_frequency', 'occupation', 'coverage_level']

Preprocessed dataset:
```

	age	bmi	children	charges	gender_female	gender_male	smoker_no	smoker_yes	region_Unknown	region_northeast	...	exercise_frequency_Unknown	occupation_Bl coll
0	46	21.45	5	20460.307669	0	1	0	1	0	0	...	0	
1	25	25.38	2	20390.899218	1	0	0	1	0	0	...	0	
2	38	44.88	2	20204.476302	0	1	0	1	0	0	...	0	
3	25	19.89	0	11789.029843	0	1	1	0	0	0	...	0	
4	49	38.21	3	19268.309838	0	1	0	1	0	0	...	0	

5 rows × 35 columns

```
Encoded dataset shape: (154292, 35)
```

```
df.to_csv("insurance_cleaned.csv", index=False)

df_encoded.to_csv("insurance_preprocessed.csv", index=False)

print("Cleaned dataset saved successfully.")
print("Preprocessed dataset saved successfully.")
```

```
Cleaned dataset saved successfully.
Preprocessed dataset saved successfully.
```

```
print("Summary Statistics:")
display(df[numerical_columns].describe())
```

Summary Statistics:

	age	bmi	children	charges
count	154292.000000	154292.000000	154292.000000	154292.000000
mean	41.472909	33.993100	2.497187	16722.247481
std	13.846492	9.249108	1.707906	4404.829719
min	18.000000	18.000000	0.000000	3445.011643
25%	29.000000	25.960000	1.000000	13603.564470
50%	41.000000	33.980000	2.000000	16604.913162
75%	53.000000	41.990000	4.000000	19754.999712
max	65.000000	50.000000	5.000000	32087.056686

```
print("Mean:")
print(df[["bmi", "charges"]].mean())
```

```
Mean:
bmi           33.993100
charges    16722.247481
dtype: float64
```

```
print("Median:")
print(df[["bmi", "charges"]].median())
```

```
Median:
bmi           33.980000
charges    16604.913162
dtype: float64
```

```
print("Mode:")

mode_df = df.drop(columns=["children"], errors="ignore").mode().iloc[[0]]

# Remove columns whose mode is 0 or 0.0
mode_df = mode_df.loc[:, ~mode_df.iloc[0].isin([0, 0.0])]

display(mode_df)
```

Mode:

	age	bmi	charges	gender_male	smoker_yes
0	43.0	39.68	16604.913162	1.0	1.0

```
print("Variance:")
print(df[["bmi", "charges"]].var())
```

```
Variance:
bmi        8.554599e+01
charges    1.940252e+07
dtype: float64
```

```
print("Standard Deviation:")
print(df[["bmi", "charges"]].std())
```

```
Standard Deviation:
bmi           9.249108
charges    4404.829719
dtype: float64
```

```
print("Quartiles:")

display(
    df[["bmi", "charges"]].quantile([0.25, 0.50, 0.75])
)
```

```
Quartiles:
```

	bmi	charges
0.25	25.96	13603.564470
0.50	33.98	16604.913162
0.75	41.99	19754.999712

```
print("Minimum values:")
print(df[["age", "bmi", "charges"]].min())

print("\nMaximum values:")
print(df[["age", "bmi", "charges"]].max())
```

```
Minimum values:
age          18.000000
bmi          18.000000
charges    3445.011643
dtype: float64

Maximum values:
age           65.000000
bmi           50.000000
charges    32087.056686
dtype: float64
```

```
print("Correlation Matrix:")
```

```
display(df[["bmi", "charges"]].corr())
```

Correlation Matrix:

	bmi	charges
bmi	1.000000	0.102837
charges	0.102837	1.000000

```
for col in categorical_columns:
    print("\nFrequency distribution of", col)
    print(df[col].value_counts())
```

```
Frequency distribution of gender
gender
male      77215
female    77077
Name: count, dtype: int64

Frequency distribution of smoker
smoker
yes    77286
no     77006
Name: count, dtype: int64

Frequency distribution of region
region
northwest    38700
northeast    38680
southwest    38559
southeast    38352
Unknown          1
Name: count, dtype: int64

Frequency distribution of medical_history
medical_history
Unknown                38942
High blood pressure    38725
Heart disease          38328
Diabetes               38297
Name: count, dtype: int64

Frequency distribution of family_medical_history
family_medical_history
Unknown                38665
High blood pressure    38576
Diabetes               38549
Heart disease          38502
Name: count, dtype: int64

Frequency distribution of exercise_frequency
exercise_frequency
Frequently      38753
```

```
Occasionally    38549
Rarely          38506
Never           38483
Unknown             1
Name: count, dtype: int64

Frequency distribution of occupation
occupation
Unemployed      38859
Student         38592
Blue collar     38566
White collar    38274
Unknown             1
Name: count, dtype: int64

Frequency distribution of coverage_level
coverage_level
```

```
for col in ["bmi", "charges"]:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_bound) |
        (df[col] > upper_bound)
    ]

    print("\nColumn:", col)
    print("Lower Bound:", lower_bound)
    print("Upper Bound:", upper_bound)
    print("Number of Outliers:", len(outliers))
```

```
Column: bmi
Lower Bound: 1.9149999999999991
Upper Bound: 66.035
Number of Outliers: 0

Column: charges
Lower Bound: 4376.411608299448
Upper Bound: 28982.152573640607
Number of Outliers: 352
```

```
plot_df = df.sample(
    n=min(10000, len(df)),
    random_state=42
)
```

```
print("Sample shape:", plot_df.shape)
```

```
Sample shape: (10000, 12)
```

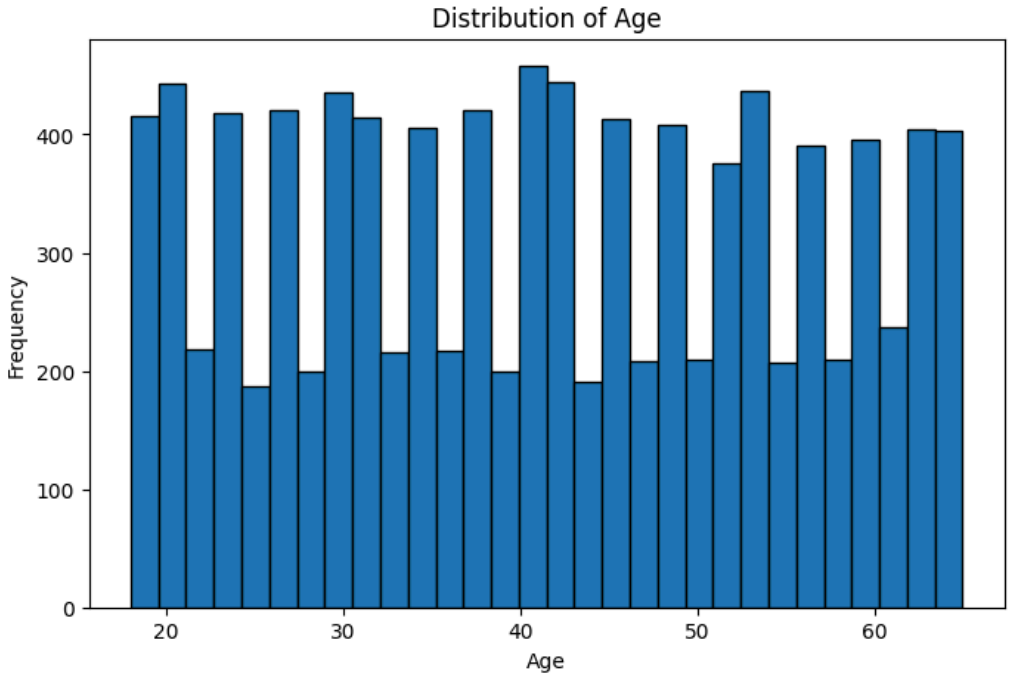
```
# Ensure plot_df is defined by sampling the dataset
plot_df = df.sample(
    n=min(10000, len(df)),
    random_state=42
)

plt.figure(figsize=(8, 5))

plt.hist(
    plot_df["age"],
    bins=30,
    edgecolor="black"
)

plt.title("Distribution of Age")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
```

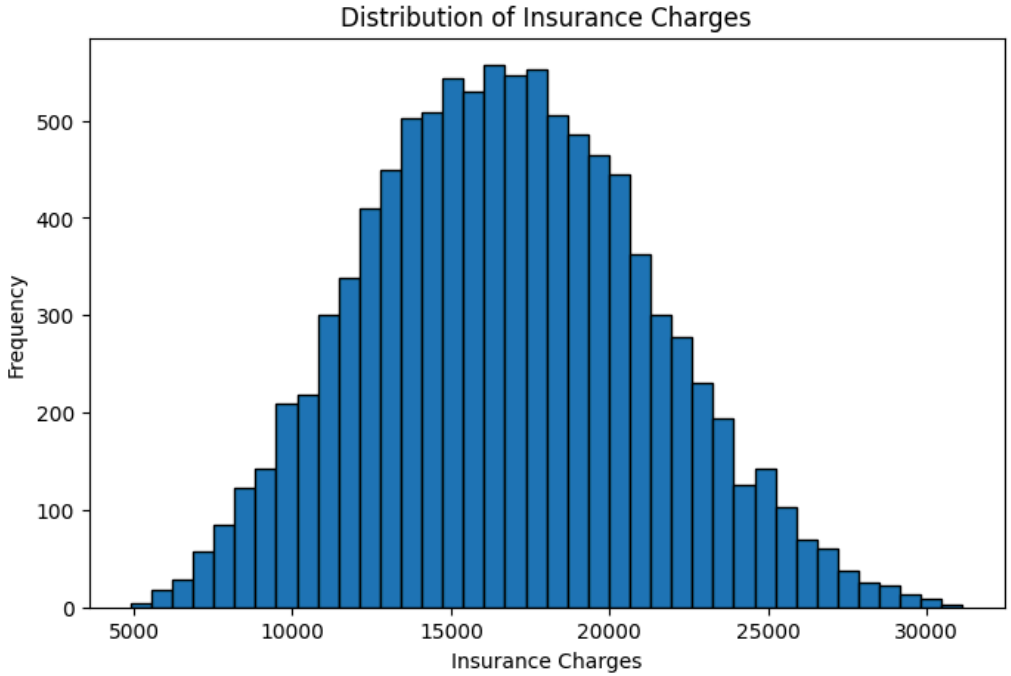


```
plt.figure(figsize=(8, 5))

plt.hist(
    plot_df["charges"],
    bins=40,
    edgecolor="black"
)

plt.title("Distribution of Insurance Charges")
plt.xlabel("Insurance Charges")
plt.ylabel("Frequency")

plt.show()
```

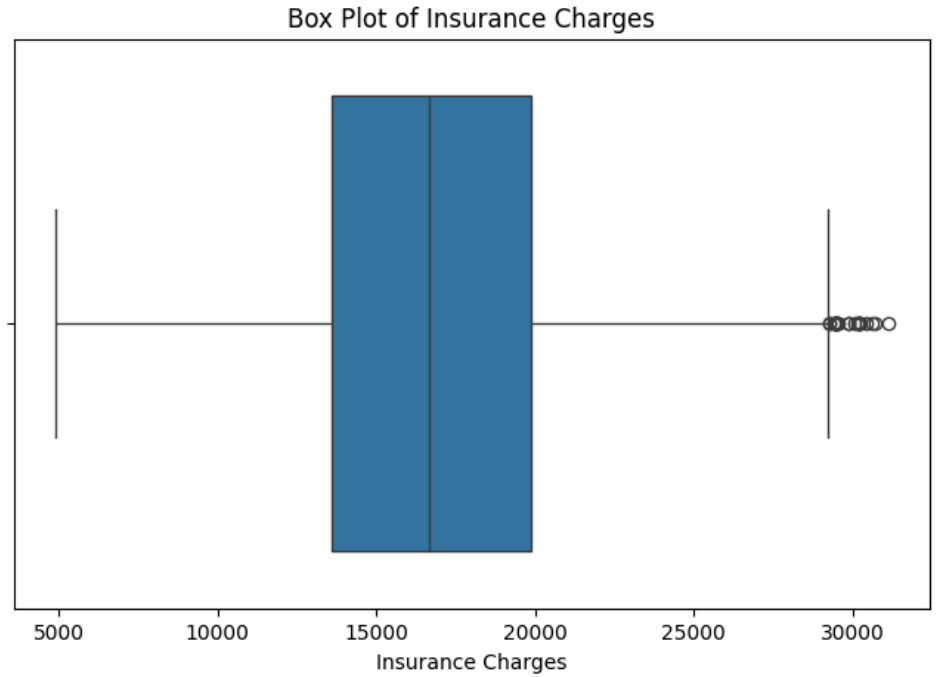



```
plt.figure(figsize=(8, 5))

sns.boxplot(x=plot_df["charges"])

plt.title("Box Plot of Insurance Charges")
plt.xlabel("Insurance Charges")

plt.show()
```



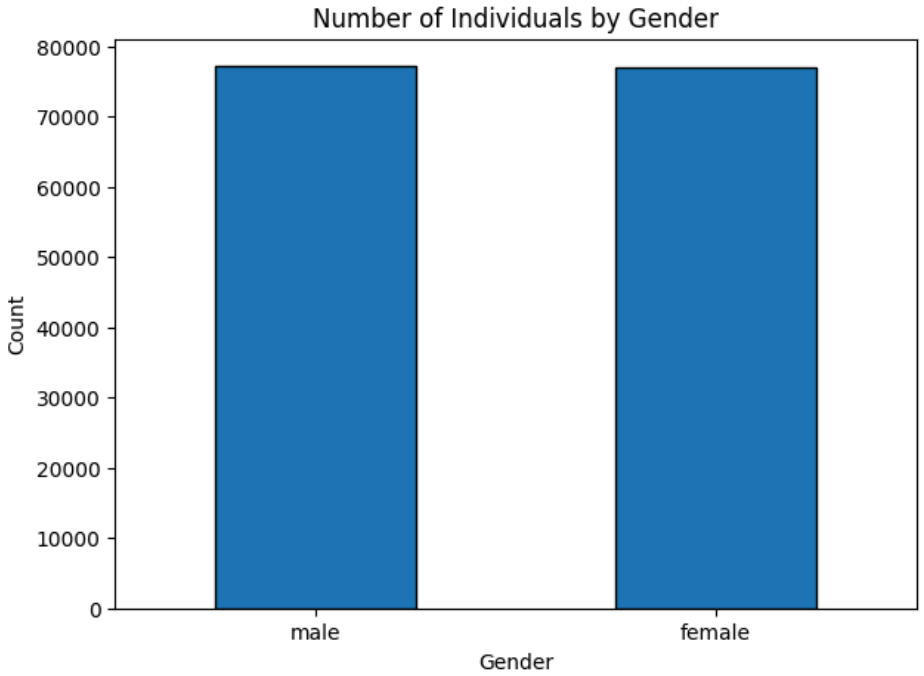

```
gender_counts = df["gender"].value_counts()

plt.figure(figsize=(7, 5))

gender_counts.plot(
    kind="bar",
    edgecolor="black"
)

plt.title("Number of Individuals by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")

plt.xticks(rotation=0)
plt.show()
```



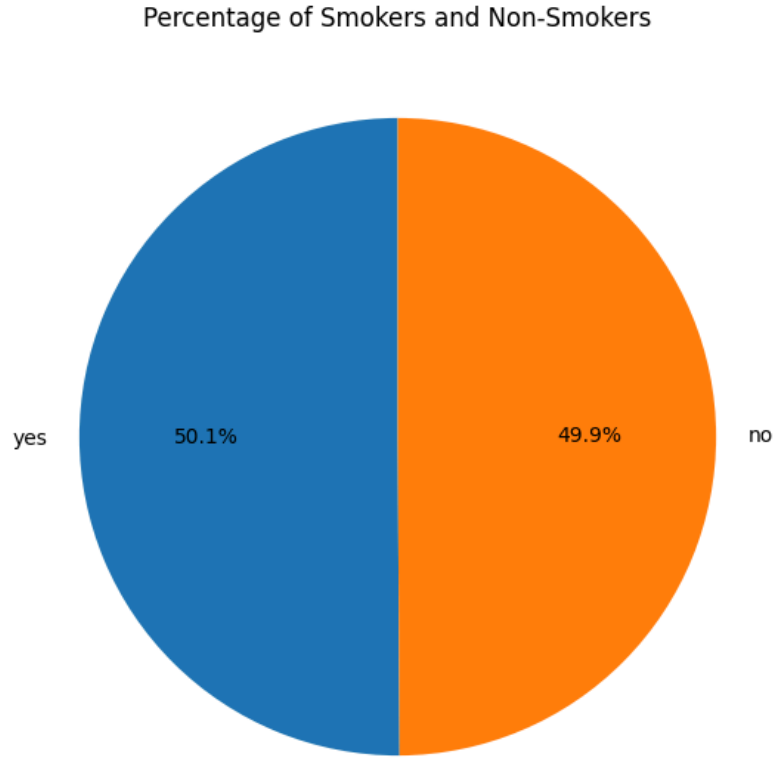

```
smoker_counts = df["smoker"].value_counts()

plt.figure(figsize=(7, 7))

plt.pie(
    smoker_counts,
    labels=smoker_counts.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Percentage of Smokers and Non-Smokers")

plt.show()
```

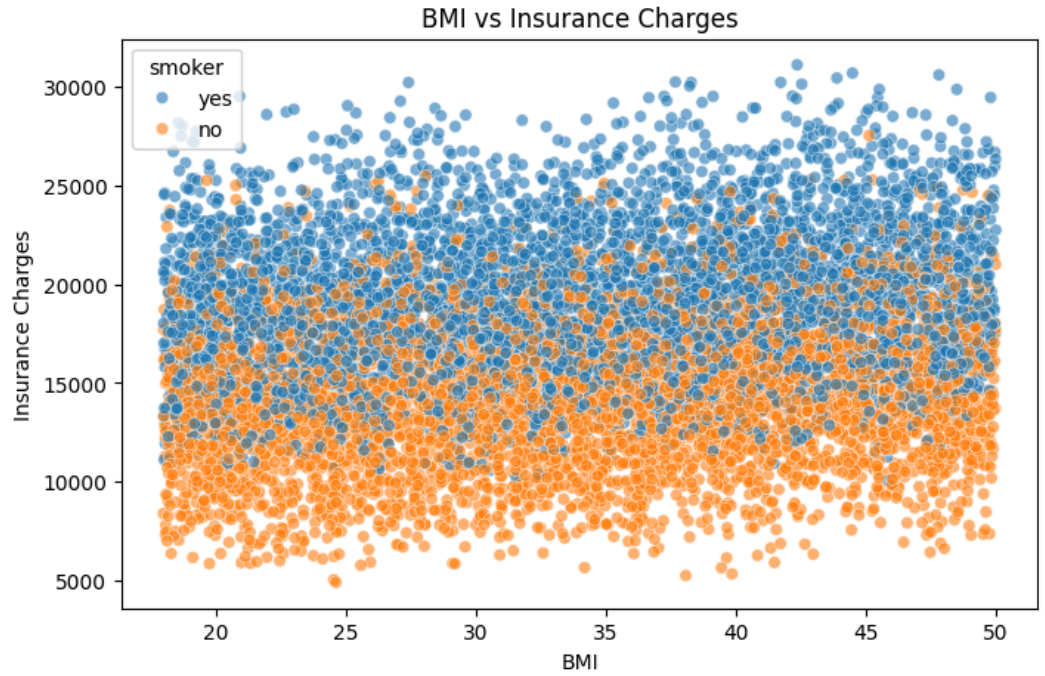



```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=plot_df,
    x="bmi",
    y="charges",
    hue="smoker",
    alpha=0.6
)

plt.title("BMI vs Insurance Charges")
plt.xlabel("BMI")
plt.ylabel("Insurance Charges")

plt.show()
```



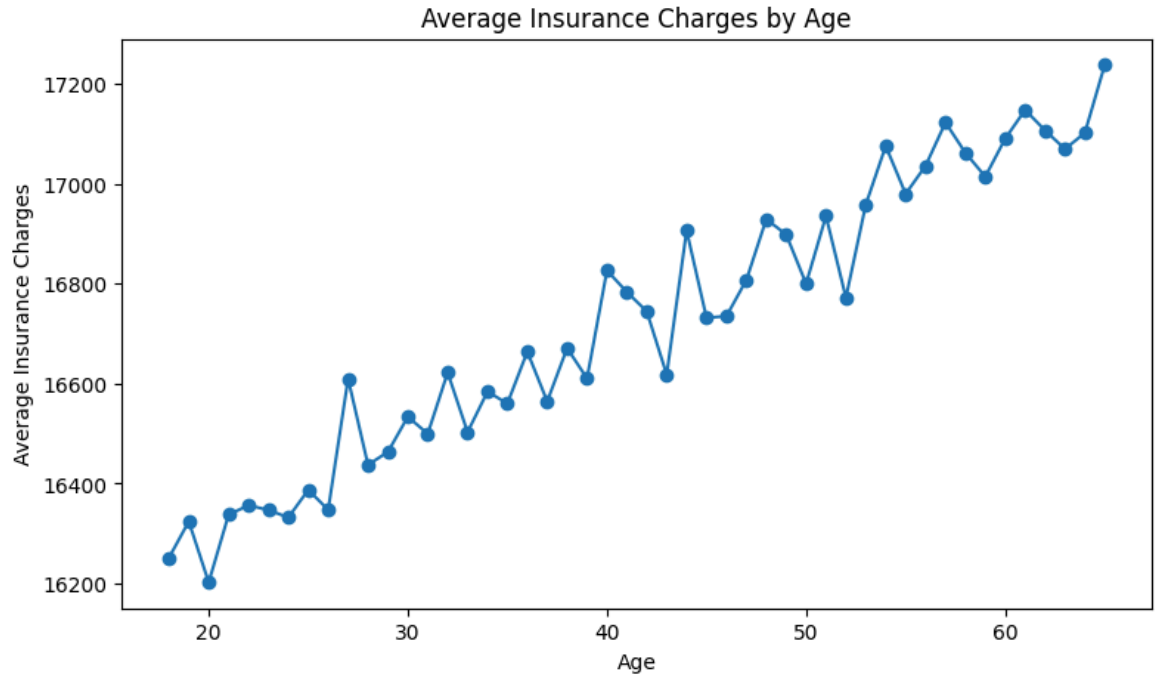

```
age_charges = df.groupby("age")["charges"].mean()

plt.figure(figsize=(9, 5))

plt.plot(
    age_charges.index,
    age_charges.values,
    marker="o"
)

plt.title("Average Insurance Charges by Age")
plt.xlabel("Age")
plt.ylabel("Average Insurance Charges")

plt.show()
```

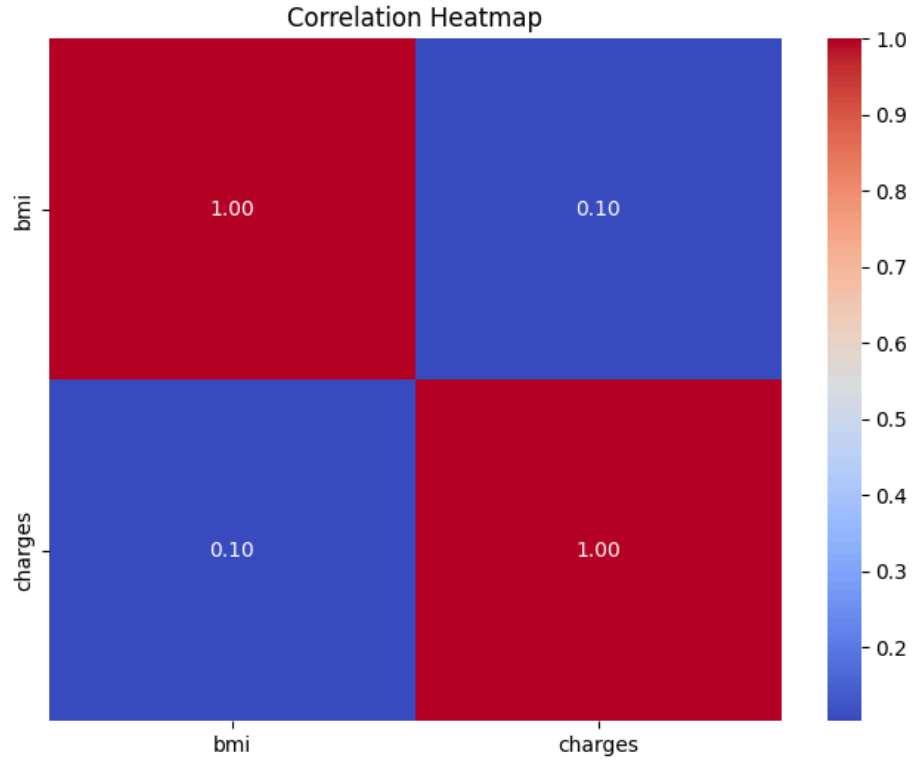



```
plt.figure(figsize=(8, 6))

correlation = df[["bmi", "charges"]].corr()

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()
```



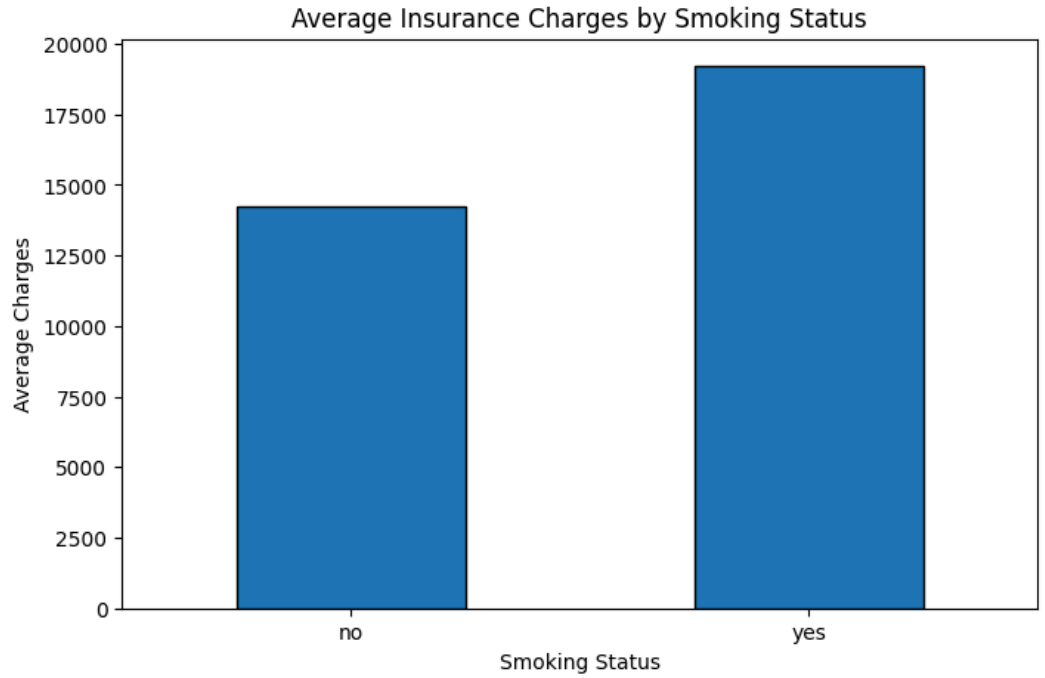

```
smoker_charges = df.groupby("smoker")["charges"].mean()

plt.figure(figsize=(8, 5))

smoker_charges.plot(
    kind="bar",
    edgecolor="black"
)

plt.title("Average Insurance Charges by Smoking Status")
plt.xlabel("Smoking Status")
plt.ylabel("Average Charges")

plt.xticks(rotation=0)
plt.show()
```



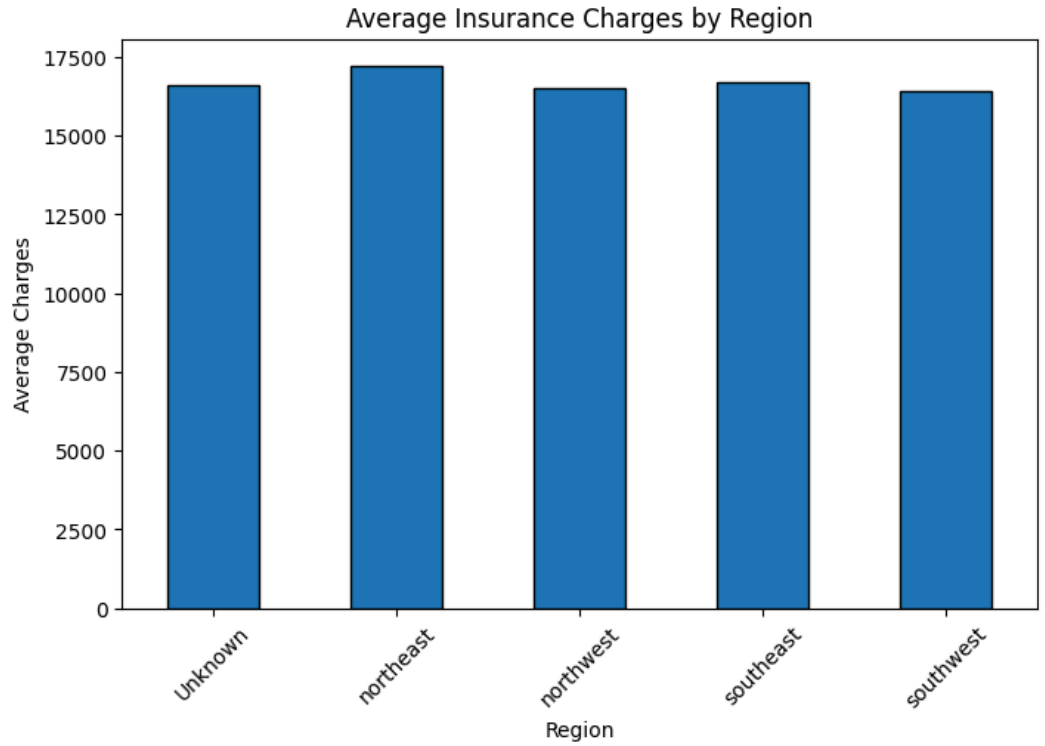

```
region_charges = df.groupby("region")["charges"].mean()

plt.figure(figsize=(8, 5))

region_charges.plot(
    kind="bar",
    edgecolor="black"
)

plt.title("Average Insurance Charges by Region")
plt.xlabel("Region")
plt.ylabel("Average Charges")

plt.xticks(rotation=45)
plt.show()
```




```
print("DATA ANALYSIS COMPLETED SUCCESSFULLY")

print("\nFinal dataset shape:")
print(df.shape)

print("\nFinal summary statistics:")
display(df[numerical_columns].describe())

print("\nGenerated visualizations:")
print("1. Histogram of age")
print("2. Histogram of insurance charges")
print("3. Box plot of insurance charges")
print("4. Bar chart of gender")
print("5. Pie chart of smoking status")
print("6. Scatter plot of BMI vs charges")
print("7. Line chart of average charges by age")
print("8. Correlation heatmap")
print("9. Average charges by smoking status")
print("10. Average charges by region")
```

```
DATA ANALYSIS COMPLETED SUCCESSFULLY

Final dataset shape:
(154292, 12)

Final summary statistics:
```

	age	bmi	children	charges
count	154292.000000	154292.000000	154292.000000	154292.000000
mean	41.472909	33.993100	2.497187	16722.247481
std	13.846492	9.249108	1.707906	4404.829719
min	18.000000	18.000000	0.000000	3445.011643
25%	29.000000	25.960000	1.000000	13603.564470
50%	41.000000	33.980000	2.000000	16604.913162
75%	53.000000	41.990000	4.000000	19754.999712
max	65.000000	50.000000	5.000000	32087.056686

```
Generated visualizations:
1. Histogram of age
2. Histogram of insurance charges
3. Box plot of insurance charges
4. Bar chart of gender
5. Pie chart of smoking status
6. Scatter plot of BMI vs charges
7. Line chart of average charges by age
8. Correlation heatmap
9. Average charges by smoking status
10. Average charges by region
```

```
# STATISTICAL ANALYSIS - INSURANCE DATASET

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
from scipy.stats import (
    chi2_contingency,
    f_oneway,
    pearsonr,
    spearmanr
)

from IPython.display import display
```

```
stats_df = df.copy()

for col in ["age", "bmi", "children", "charges"]:
    stats_df[col] = pd.to_numeric(
        stats_df[col], errors="coerce"
    )

stats_df = stats_df.dropna(
    subset=["age", "bmi", "children", "charges"]
)

analysis_df = stats_df.sample(
    n=min(50000, len(stats_df)),
    random_state=42
)

bootstrap_df = stats_df.sample(
    n=min(10000, len(stats_df)),
    random_state=42
)

print("Full dataset shape:", stats_df.shape)
print("Analysis sample shape:", analysis_df.shape)
print("Bootstrap sample shape:", bootstrap_df.shape)

print("\nNumerical summary:")
display(stats_df[["age", "bmi", "children", "charges"]].describe())
```

```
Full dataset shape: (154292, 35)
Analysis sample shape: (50000, 35)
Bootstrap sample shape: (10000, 35)

Numerical summary:
```

	age	bmi	children	charges
count	154292.000000	154292.000000	154292.000000	154292.000000
mean	41.472909	33.993100	2.497187	16722.247481
std	13.846492	9.249108	1.707906	4404.829719
min	18.000000	18.000000	0.000000	3445.011643
25%	29.000000	25.960000	1.000000	13603.564470
50%	41.000000	33.980000	2.000000	16604.913162
75%	53.000000	41.990000	4.000000	19754.999712
max	65.000000	50.000000	5.000000	32087.056686

```
charges = stats_df["charges"].to_numpy()
```

```
population_mean = np.mean(charges)
population_sd = np.std(charges, ddof=1)

print("Reference mean of charges:", round(population_mean, 2))
print("Reference standard deviation:", round(population_sd, 2))

rng = np.random.default_rng(42)

sample_size = 100
number_of_samples = 1000

sample_means = np.array([
    np.mean(rng.choice(charges, size=sample_size, replace=True))
    for _ in range(number_of_samples)
])

print("\nMean of sample means:", round(sample_means.mean(), 2))
print("Expected standard error:",
      round(population_sd / np.sqrt(sample_size), 2))
print("Observed standard deviation of sample means:",
      round(sample_means.std(ddof=1), 2))

plt.figure(figsize=(9, 5))
sns.histplot(sample_means, bins=30, kde=True)
plt.axvline(population_mean, linestyle="--",
            label="Full dataset mean")
plt.title("CLT: Sampling Distribution of Mean Insurance Charges")
plt.xlabel("Sample mean of charges")
plt.ylabel("Frequency")
plt.legend()
plt.show()

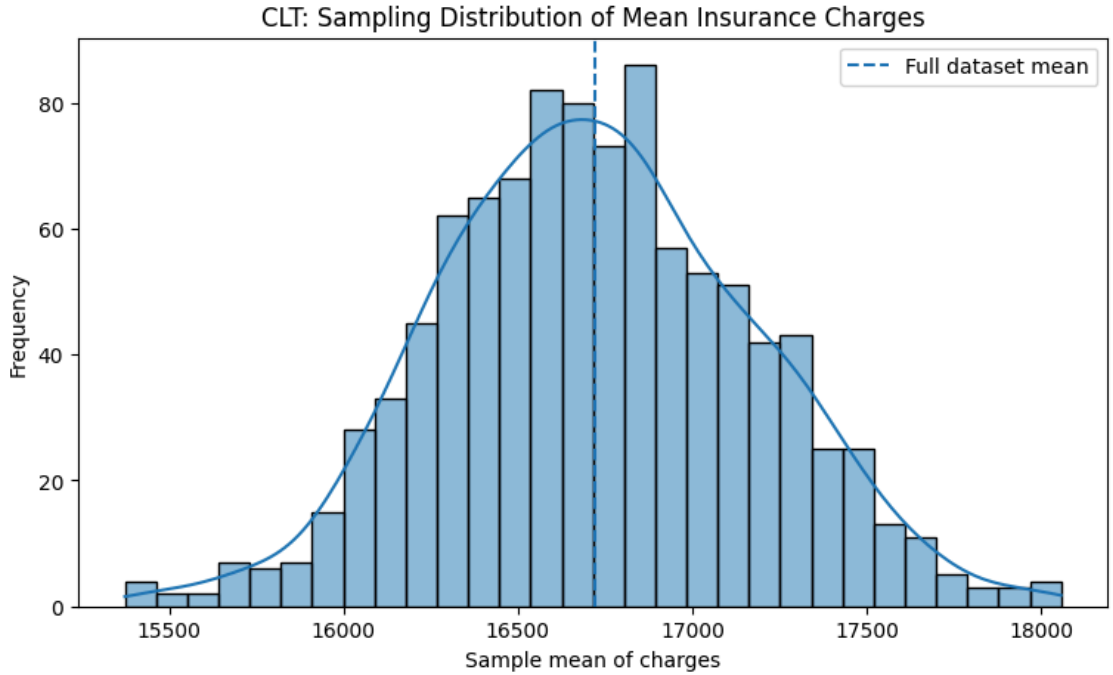
n = len(charges)
mean_charge = np.mean(charges)
se = stats.sem(charges)

ci_low, ci_high = stats.norm.interval(
    0.95, loc=mean_charge, scale=se
)

print("\n95% Confidence Interval for Mean Charges")
print("Sample mean:", round(mean_charge, 2))
print("Lower bound:", round(ci_low, 2))
print("Upper bound:", round(ci_high, 2))
```

```
Reference mean of charges: 16722.25
Reference standard deviation: 4404.83

Mean of sample means: 16723.57
Expected standard error: 440.48
Observed standard deviation of sample means: 450.17
```




```
95% Confidence Interval for Mean Charges
Sample mean: 16722.25
Lower bound: 16700.27
Upper bound: 16744.23
```

```
# BOOTSTRAP ESTIMATION

boot_charges = bootstrap_df["charges"].to_numpy()

rng = np.random.default_rng(42)
B = 2000
n_boot = len(boot_charges)

bootstrap_means = np.empty(B)
bootstrap_medians = np.empty(B)

for i in range(B):
    sample = rng.choice(
        boot_charges,
```

```
            size=n_boot,
            replace=True
        )

        bootstrap_means[i] = np.mean(sample)
        bootstrap_medians[i] = np.median(sample)

mean_ci = np.percentile(bootstrap_means, [2.5, 97.5])
median_ci = np.percentile(bootstrap_medians, [2.5, 97.5])

print("BOOTSTRAP RESULTS")
print("-" * 40)

print("Original sample mean:", round(np.mean(boot_charges), 2))
print("Bootstrap mean estimate:", round(bootstrap_means.mean(), 2))
print("Mean 95% CI:", np.round(mean_ci, 2))

print("\nOriginal sample median:", round(np.median(boot_charges), 2))
print("Bootstrap median estimate:", round(bootstrap_medians.mean(), 2))
print("Median 95% CI:", np.round(median_ci, 2))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

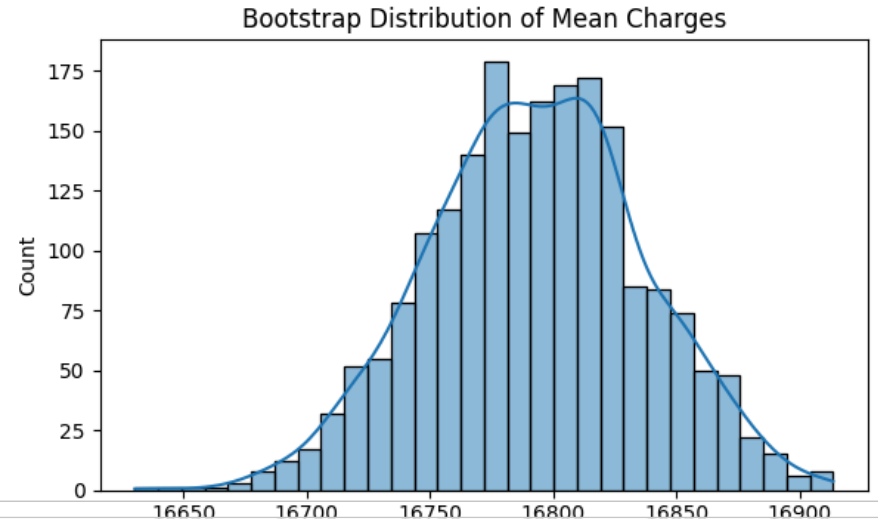
sns.histplot(bootstrap_means, bins=30, kde=True, ax=axes[0])
axes[0].set_title("Bootstrap Distribution of Mean Charges")
axes[0].set_xlabel("Bootstrap mean")

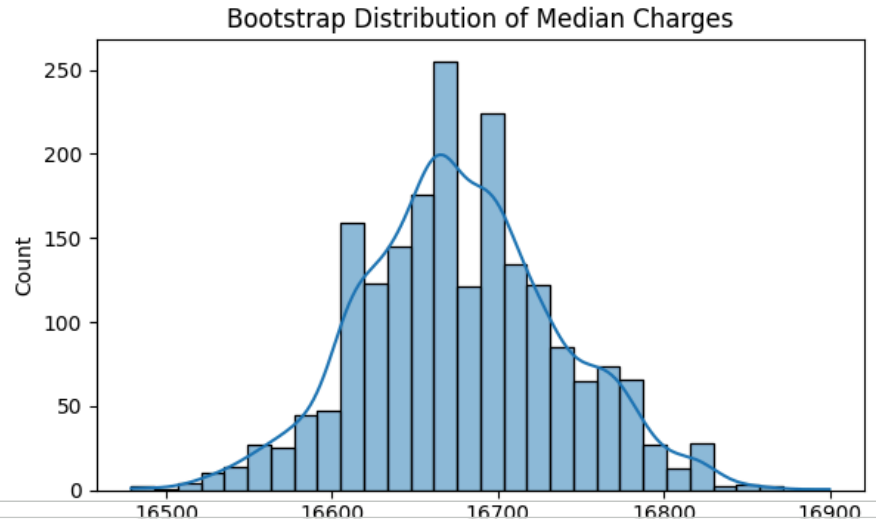
sns.histplot(bootstrap_medians, bins=30, kde=True, ax=axes[1])
axes[1].set_title("Bootstrap Distribution of Median Charges")
axes[1].set_xlabel("Bootstrap median")

plt.tight_layout()
plt.show()
```

```
BOOTSTRAP RESULTS
----------------------------------------
Original sample mean: 16792.82
Bootstrap mean estimate: 16792.74
Mean 95% CI: [16707.71 16876.13]

Original sample median: 16670.36
Bootstrap median estimate: 16677.83
Median 95% CI: [16561.55 16797.47]
```






```
# SKEWNESS AND MEAN VS MEDIAN

charges = analysis_df["charges"]

mean_value = charges.mean()
median_value = charges.median()
skewness_value = stats.skew(charges, bias=False)

print("Mean charges:", round(mean_value, 2))
print("Median charges:", round(median_value, 2))
print("Skewness:", round(skewness_value, 4))

if skewness_value > 0:
    print("Distribution is positively (right) skewed.")
elif skewness_value < 0:
    print("Distribution is negatively (left) skewed.")
else:
    print("Distribution has approximately zero skewness.")

if mean_value > median_value:
    print("Mean > Median: consistent with right-skewness.")
elif mean_value < median_value:
    print("Mean < Median: consistent with left-skewness.")
```

```
else:
    print("Mean and median are approximately equal.")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(charges, bins=50, kde=True, ax=axes[0])
axes[0].axvline(mean_value, linestyle="--", label="Mean")
axes[0].axvline(median_value, linestyle=":", label="Median")
axes[0].set_title("Distribution of Insurance Charges")
axes[0].legend()

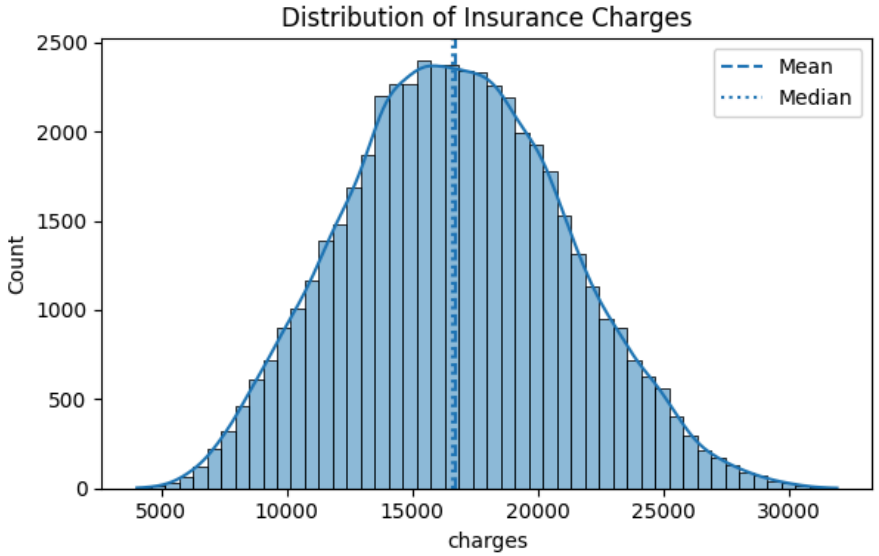
sns.histplot(sample_means, bins=30, kde=True, ax=axes[1])
axes[1].set_title("Sampling Distribution of Mean Charges")

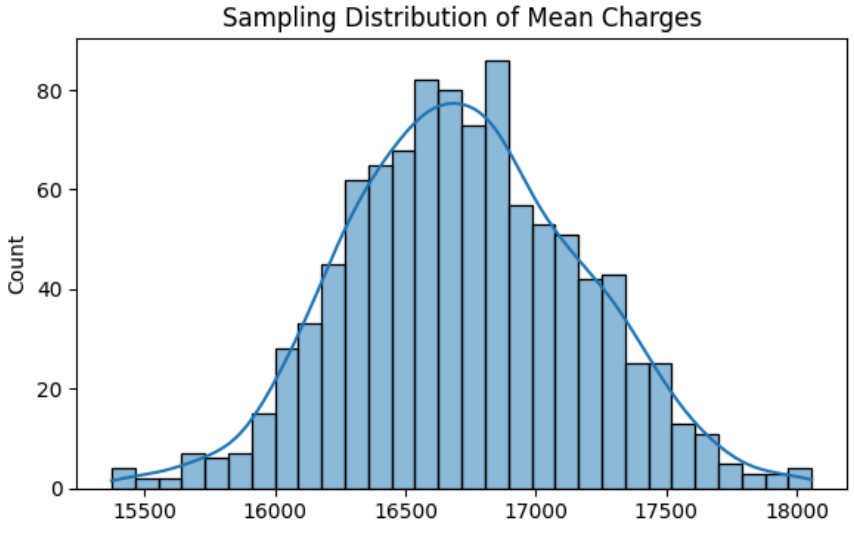
plt.tight_layout()
plt.show()

comparison = analysis_df.groupby("smoker")["charges"].agg(
    ["mean", "median", "std", "count"]
)

print("\nMean and median charges by smoking status:")
display(comparison)
```

```
Mean charges: 16723.73
Median charges: 16624.63
Skewness: 0.1289
Distribution is positively (right) skewed.
Mean > Median: consistent with right-skewness.
```





Mean and median charges by smoking status:

smoker	mean	median	std	count
no	14222.201728	14088.049555	3638.738083	24941

```
# ONE-SAMPLE Z-TEST AND T-TEST

mu0 = 17000
alpha = 0.05

test_data = analysis_df["charges"].dropna().to_numpy()

n = len(test_data)
sample_mean = np.mean(test_data)
sample_sd = np.std(test_data, ddof=1)
standard_error = sample_sd / np.sqrt(n)


z_stat = (sample_mean - mu0) / standard_error
z_p_value = 2 * stats.norm.sf(abs(z_stat))

t_stat, t_p_value = stats.ttest_1samp(
    test_data, popmean=mu0
```

```
)

print("ONE-SAMPLE Z-TEST")
print("Null mean:", mu0)
print("Sample mean:", round(sample_mean, 2))
print("Z statistic:", round(z_stat, 4))
print("P-value:", z_p_value)

if z_p_value < alpha:
    print("Decision: Reject H0.")
else:
    print("Decision: Fail to reject H0.")

print("\nONE-SAMPLE T-TEST")
print("T statistic:", round(t_stat, 4))
print("Degrees of freedom:", n - 1)
print("P-value:", t_p_value)

if t_p_value < alpha:
    print("Decision: Reject H0.")
else:
    print("Decision: Fail to reject H0.")
```

```
ONE-SAMPLE Z-TEST
Null mean: 17000
Sample mean: 16723.73
Z statistic: -13.9983
P-value: 1.5956444415755626e-44
Decision: Reject H0.

ONE-SAMPLE T-TEST
T statistic: -13.9983
Degrees of freedom: 49999
P-value: 1.9361877719817866e-44
Decision: Reject H0.
```

```
# CHI-SQUARE TEST OF INDEPENDENCe

alpha = 0.05

contingency_table = pd.crosstab(
    analysis_df["smoker"],
    analysis_df["coverage_level"]
)

print("Observed contingency table:")
display(contingency_table)

chi2_stat, chi2_p, degrees_freedom, expected = chi2_contingency(
    contingency_table
)
```

```
expected_table = pd.DataFrame(
    expected,
    index=contingency_table.index,
    columns=contingency_table.columns
)

print("\nExpected frequencies under independence:")
display(expected_table.round(2))

print("\nChi-square statistic:", round(chi2_stat, 4))
print("Degrees of freedom:", degrees_freedom)
print("P-value:", chi2_p)

if chi2_p < alpha:
    print("Decision: Reject H0.")
    print("Conclusion: Smoking status and coverage level are associated.")
else:
    print("Decision: Fail to reject H0.")
    print("Conclusion: Insufficient evidence of an association.")

# Visualize observed counts
plt.figure(figsize=(8, 5))
sns.heatmap(contingency_table, annot=True, fmt="d", cmap="Blues")
plt.title("Smoking Status vs Coverage Level")
plt.xlabel("Coverage Level")
plt.ylabel("Smoking Status")
plt.show()
```

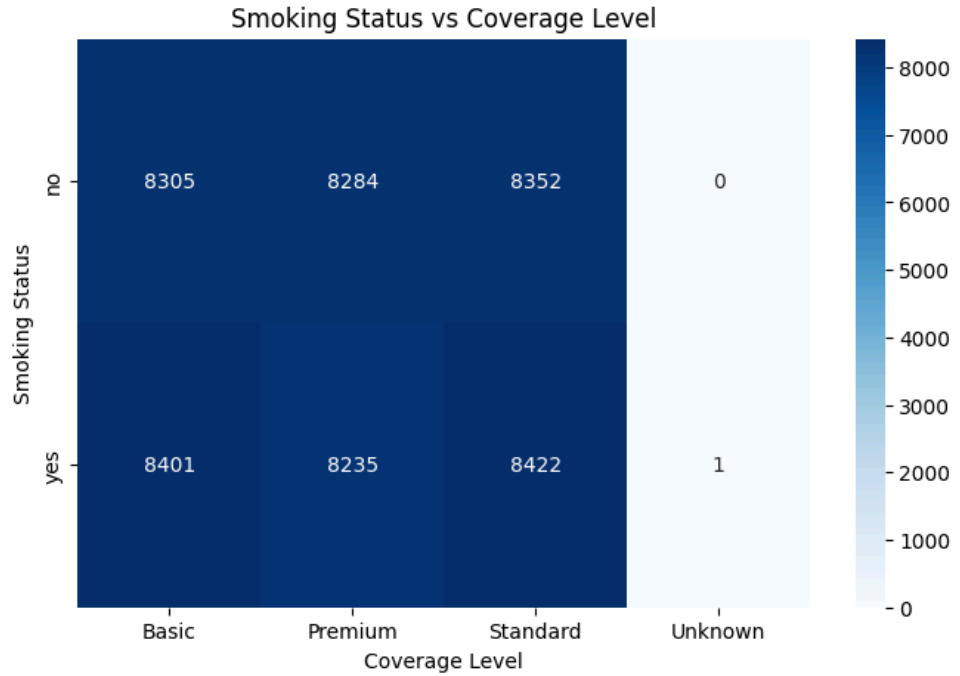
```
Observed contingency table:
```

coverage_level	Basic	Premium	Standard	Unknown
smoker				
no	8305	8284	8352	0
yes	8401	8235	8422	1

```
Expected frequencies under independence:
```

coverage_level	Basic	Premium	Standard	Unknown
smoker				
no	8333.29	8240.01	8367.21	0.5
yes	8372.71	8278.99	8406.79	0.5

```
Chi-square statistic: 1.7107
Degrees of freedom: 3
P-value: 0.6345677374264138
Decision: Fail to reject H0.
Conclusion: Insufficient evidence of an association.
```




```
# ONE-WAY ANOVA
```

```
alpha = 0.05

groups = {
    level: group["charges"].to_numpy()
    for level, group in analysis_df.groupby("coverage_level")
}

print("Number of observations per group:")
for level, values in groups.items():
    print(level, ":", len(values))

print("\nMean charges by coverage level:")
display(
    analysis_df.groupby("coverage_level")["charges"]
    .agg(["mean", "median", "std", "count"])
)

f_stat, anova_p = f_oneway(*groups.values())

print("\nANOVA F-statistic:", round(f_stat, 4))
print("P-value:", anova_p)

if anova_p < alpha:
    print("Decision: Reject H0.")
    print("Conclusion: At least one group mean differs.")
else:
    print("Decision: Fail to reject H0.")
    print("Conclusion: Insufficient evidence of different group means.")

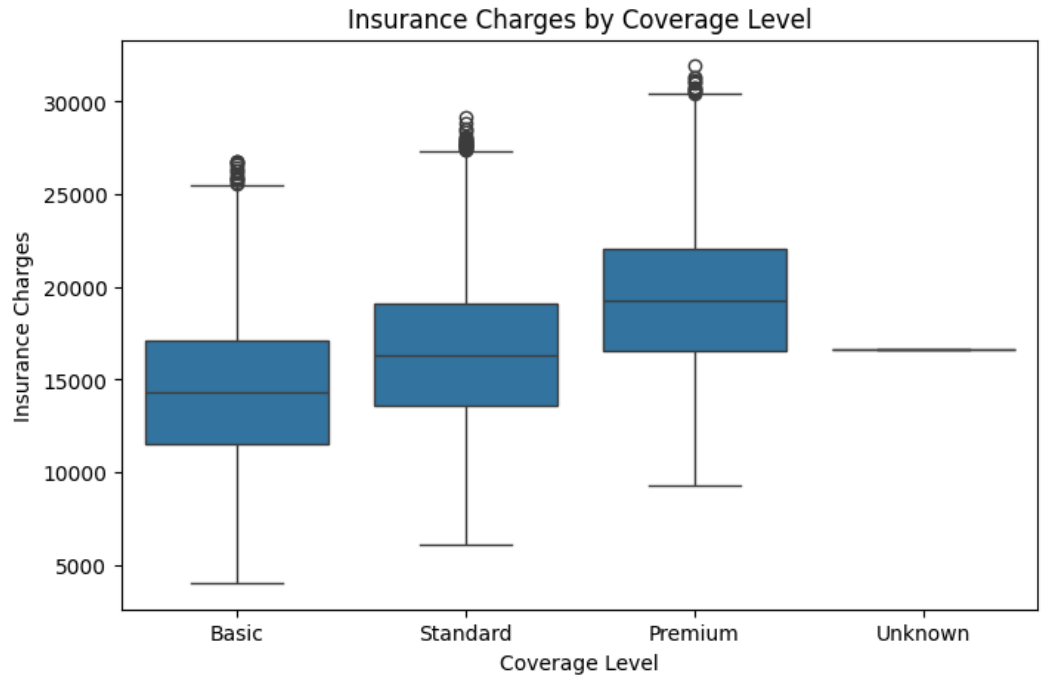
plt.figure(figsize=(8, 5))
sns.boxplot(data=analysis_df, x="coverage_level", y="charges")
plt.title("Insurance Charges by Coverage Level")
plt.xlabel("Coverage Level")
plt.ylabel("Insurance Charges")
plt.show()
```

```
Number of observations per group:
Basic : 16706
Premium : 16519
Standard : 16774
Unknown : 1

Mean charges by coverage level:
```

coverage_level	mean	median	std	count
Basic	14403.154112	14314.745433	3925.526976	16706
Premium	19385.298735	19294.832279	3907.890563	16519
Standard	16413.786338	16350.871237	3902.956636	16774
Unknown	16604.913162	16604.913162	NaN	1

```
ANOVA F-statistic: 4543.098
P-value: 0.0
Decision: Reject H0.
Conclusion: At least one group mean differs.
```




```
numeric_cols = ["bmi", "charges"]

numeric_data = analysis_df[numeric_cols]
```

```
covariance_matrix = numeric_data.cov()

print("COVARIANCE MATRIX")
display(covariance_matrix.round(3))

correlation_matrix = numeric_data.corr(method="pearson")

print("\nPEARSON CORRELATION MATRIX")
display(correlation_matrix.round(4))

plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".3f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1
)

plt.title("Correlation Matrix of Insurance Variables")
plt.show()

bmi_cov = numeric_data["bmi"].cov(numeric_data["charges"])
bmi_corr = numeric_data["bmi"].corr(numeric_data["charges"])

print("\nBMI vs Insurance Charges")
print("Covariance:", round(bmi_cov, 3))
print("Pearson correlation:", round(bmi_corr, 4))

if bmi_corr > 0:
    print("Direction: Positive relationship.")
elif bmi_corr < 0:
    print("Direction: Negative relationship.")
else:
    print("Direction: No linear relationship.")

if abs(bmi_corr) < 0.3:
    print("Strength: Weak linear relationship.")
elif abs(bmi_corr) < 0.7:
    print("Strength: Moderate linear relationship.")
else:
    print("Strength: Strong linear relationship.")

plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=analysis_df,
    x="bmi",
    y="charges",
    alpha=0.35
)
```

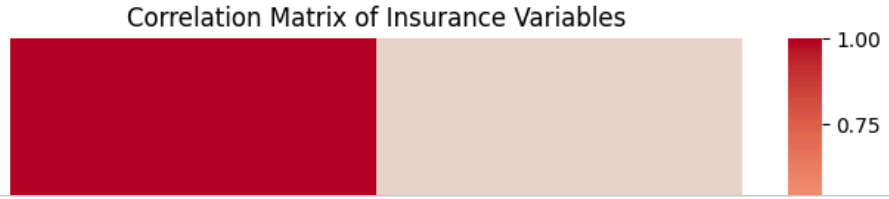
```
plt.title("BMI vs Insurance Charges")
plt.show()
```

```
COVARIANCE MATRIX
```

	bmi	charges
bmi	85.247	4.195351e+03
charges	4195.351	1.947589e+07

```
PEARSON CORRELATION MATRIX
```

	bmi	charges
bmi	1.000	0.103
charges	0.103	1.000




```
# PEARSON AND SPEARMAN CORRELATION TESTS

alpha = 0.05

x = analysis_df["bmi"].to_numpy()
y = analysis_df["charges"].to_numpy()

pearson_r, pearson_p = pearsonr(x, y)

spearman_rho, spearman_p = spearmanr(x, y)

def describe_strength(value):
    absolute_value = abs(value)

    if absolute_value < 0.3:
        strength = "Weak"
    elif absolute_value < 0.7:
        strength = "Moderate"
    else:
        strength = "Strong"

    direction = (
        "positive" if value > 0
        else "negative" if value < 0
        else "no"
    )

    return f"{strength}, {direction} association"

print("RELATIONSHIP: BMI vs INSURANCE CHARGES")
```

```
print("=" * 45)

print("\nPEARSON CORRELATION")
print("Coefficient:", round(pearson_r, 4))
print("Interpretation:", describe_strength(pearson_r))
print("P-value:", pearson_p)

if pearson_p < alpha:
    print("Decision: Reject H0; evidence of a linear relationship.")
else:
    print("Decision: Fail to reject H0; insufficient evidence of a linear relationship.")

print("\nSPEARMAN CORRELATION")
print("Coefficient:", round(spearman_rho, 4))
print("Interpretation:", describe_strength(spearman_rho))
print("P-value:", spearman_p)

if spearman_p < alpha:
    print("Decision: Reject H0; evidence of a monotonic relationship.")
else:
    print("Decision: Fail to reject H0; insufficient evidence of a monotonic relationship.")
```

```
RELATIONSHIP: BMI vs INSURANCE CHARGES
==============================================

PEARSON CORRELATION
Coefficient: 0.103
Interpretation: Weak, positive association
P-value: 6.707170724804e-118
Decision: Reject H0; evidence of a linear relationship.

SPEARMAN CORRELATION
Coefficient: 0.0986
Interpretation: Weak, positive association
P-value: 3.64410596950538e-108
Decision: Reject H0; evidence of a monotonic relationship.
```

```
# STATISTICAL ANALYSIS SUMMARY

print("=" * 55)
print("INSURANCE DATASET - STATISTICAL ANALYSIS SUMMARY")
print("=" * 55)

print("\n1. CLT AND CONFIDENCE INTERVAL")
print("Mean charges:", round(stats_df["charges"].mean(), 2))
print("95% CI:", round(ci_low, 2), "to", round(ci_high, 2))

print("\n2. BOOTSTRAP ESTIMATION")
print("Bootstrap mean CI:", np.round(mean_ci, 2))
print("Bootstrap median CI:", np.round(median_ci, 2))

print("\n3. MEAN VS MEDIAN")
```

```
print("Mean:", round(stats_df["charges"].mean(), 2))
print("Median:", round(stats_df["charges"].median(), 2))

print("\n4. ONE-SAMPLE TESTS")
print("Z-test p-value:", z_p_value)
print("T-test p-value:", t_p_value)

print("\n5. CHI-SQUARE TEST")
print("Chi-square p-value:", chi2_p)

print("\n6. ONE-WAY ANOVA")
print("ANOVA p-value:", anova_p)

print("\n7. CORRELATION TESTS")
print("Pearson r:", round(pearson_r, 4),
      "| p-value:", pearson_p)
print("Spearman rho:", round(spearman_rho, 4),
      "| p-value:", spearman_p)

print("\nSTATISTICAL ANALYSIS COMPLETED")
```

```
======================================================
INSURANCE DATASET - STATISTICAL ANALYSIS SUMMARY
======================================================

1. CLT AND CONFIDENCE INTERVAL
Mean charges: 16722.25
95% CI: 16700.27 to 16744.23

2. BOOTSTRAP ESTIMATION
Bootstrap mean CI: [16707.71 16876.13]
Bootstrap median CI: [16561.55 16797.47]

3. MEAN VS MEDIAN
Mean: 16722.25
Median: 16604.91

4. ONE-SAMPLE TESTS
Z-test p-value: 1.5956444415755626e-44
T-test p-value: 1.9361877719817866e-44

5. CHI-SQUARE TEST
Chi-square p-value: 0.6345677374264138

6. ONE-WAY ANOVA
ANOVA p-value: 0.0

7. CORRELATION TESTS
Pearson r: 0.103 | p-value: 6.707170724804e-118
Spearman rho: 0.0986 | p-value: 3.64410596950538e-108

STATISTICAL ANALYSIS COMPLETED
```

```
%pip install pandas numpy scikit-learn matplotlib seaborn joblib streamlit
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages (0.13.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.13/dist-packages (1.6.0)
Collecting streamlit
  Downloading streamlit-1.66.1-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.5)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.3)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib) (3.1.2)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=5.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (5.5.0)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (8.5.0)
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Downloading pydeck-0.9.3-py2.py3-none-any.whl.metadata (4.2 kB)
Requirement already satisfied: protobuf<8,>=5.26.1 in /usr/local/lib/python3.13/dist-packages (from streamlit) (6.33.6)
Requirement already satisfied: pyarrow!=25.0.0,<27,>=10.0.1 in /usr/local/lib/python3.13/dist-packages (from streamlit) (23.0.1)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (4.16.0)
Requirement already satisfied: starlette<2,>=0.46.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (1.9.0)
Requirement already satisfied: uvicorn<1,>=0.30.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.54.0)
Requirement already satisfied: anyio<5,>=4.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (4.15.1)
Requirement already satisfied: python-multipart<1,>=0.0.10 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.0.32)
Requirement already satisfied: websockets<18,>=12.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (15.0.1)
Requirement already satisfied: itsdangerous<3,>=2.1.2 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.2.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.13/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (3.1.6)
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (4.26.0)
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (2.27.1)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.13/dist-packages (from anyio<5,>=4.0.0->streamlit) (3.20)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (3.4.9)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (2026.10.19)
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.13/dist-packages (from uvicorn<1,>=0.30.0->streamlit) (0.16.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (3.0.4)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0-
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (0.37
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.13/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (2026.9.1
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading streamlit-1.66.1-py3-none-any.whl (10.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.4/10.4 MB 28.7 MB/s  0:00:00
Downloading pydeck-0.9.3-py2.py3-none-any.whl (11.4 MB)
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.4/11.4 MB 58.7 MB/s  0:00:00
Installing collected packages: pydeck, streamlit
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2/2 [streamlit]
```

```
import os
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
df = pd.read_csv("insurance_cleaned.csv")
df = pd.read_csv("insurance_preprocessed.csv")

df
```

	age	bmi	children	charges	gender_female	gender_male	smoker_no	smoker_yes	region_Unknown	region_northeast	...	exercise_frequency_Unknown	occupati
0	46	21.45	5	20460.307669	0	1	0	1	0	0	...	0	
1	25	25.38	2	20390.899218	1	0	0	1	0	0	...	0	
2	38	44.88	2	20204.476302	0	1	0	1	0	0	...	0	
3	25	19.89	0	11789.029843	0	1	1	0	0	0	...	0	
4	49	38.21	3	19268.309838	0	1	0	1	0	0	...	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
154287	34	31.96	5	18002.417360	1	0	0	1	0	0	...	0	
154288	43	22.26	0	15888.519502	1	0	1	0	0	0	...	0	
154289	25	20.31	3	12762.333419	0	1	1	0	0	0	...	0	
154290	19	45.84	0	16697.500532	0	1	1	0	0	1	...	0	
154291	25	49.12	3	16604.913162	1	0	0	1	1	0	...	1	

154292 rows × 35 columns

```
target = "charges"

X = df.drop(columns=[target])
y = df[target]

print("Input features:")
print(X.columns.tolist())

print("\nTarget variable:", target)

print("\nNumber of input features:", X.shape[1])
print("Number of records:", X.shape[0])
```

```
Input features:
['age', 'bmi', 'children', 'gender_female', 'gender_male', 'smoker_no', 'smoker_yes', 'region_Unknown', 'region_northeast', 'region_northwest', 'region_southeast', 'reg

Target variable: charges

Number of input features: 34
Number of records: 154292
```

```
numerical_features = X.select_dtypes(
    include=np.number
).columns.tolist()
```

```
categorical_features = X.select_dtypes(
    exclude=np.number
).columns.tolist()

print("Numerical features:")
print(numerical_features)

print("\nCategorical features:")
print(categorical_features)
```

```
Numerical features:
['age', 'bmi', 'children', 'gender_female', 'gender_male', 'smoker_no', 'smoker_yes', 'region_Unknown', 'region_northeast', 'region_northwest', 'region_southeast', 'reg

Categorical features:
[]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training records:", X_train.shape[0])
print("Testing records:", X_test.shape[0])
```

```
Training records: 123433
Testing records: 30859
```

```
numerical_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="median")
        ),
        (
            "scaler",
            StandardScaler()
        )
    ]
)

categorical_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="most_frequent")
        ),
        (
            "onehot",
```

```
            OneHotEncoder(handle_unknown="ignore")
        )
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            numerical_transformer,
            numerical_features
        ),
        (
            "cat",
            categorical_transformer,
            categorical_features
        )
    ]
)

print("Preprocessing pipeline created successfully.")
```

```
Preprocessing pipeline created successfully.
```

```
linear_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regressor", LinearRegression())
    ]
)

linear_model.fit(X_train, y_train)

print("Linear Regression model trained successfully.")
```

```
Linear Regression model trained successfully.
```

```
linear_predictions = linear_model.predict(X_test)

print("First 10 actual values:")
print(y_test.head(10).to_numpy())

print("\nFirst 10 predicted values:")
print(linear_predictions[:10])
```

```
First 10 actual values:
[18792.22167438 17242.75235419 21389.27975968 13178.88819901
 23361.53608451 29540.84465171 14176.24786201 20563.51728147
 25045.95188889 17366.21113544]
```

```
First 10 predicted values:
[18604.13047409 17261.72563354 21781.39816927 13386.60434416
 22944.09202953 29276.04526569 14267.72162204 20348.17555409
 24962.91159832 17329.56749547]
```

```
linear_mae = mean_absolute_error(
    y_test,
    linear_predictions
)

linear_mse = mean_squared_error(
    y_test,
    linear_predictions
)

linear_rmse = np.sqrt(linear_mse)

linear_r2 = r2_score(
    y_test,
    linear_predictions
)

print("LINEAR REGRESSION RESULTS")
print("-------------------------")
print("MAE:", round(linear_mae, 2))
print("MSE:", round(linear_mse, 2))
print("RMSE:", round(linear_rmse, 2))
print("R2 Score:", round(linear_r2, 4))
```

```
LINEAR REGRESSION RESULTS
-------------------------
MAE: 250.55
MSE: 83585.69
RMSE: 289.11
R2 Score: 0.9957
```

```
# Limit Random Forest training data for performance
rf_sample_size = min(100_000, len(X_train))

X_rf_train = X_train.sample(
    n=rf_sample_size,
    random_state=42
)

y_rf_train = y_train.loc[X_rf_train.index]

rf_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "regressor",
```

```
            RandomForestRegressor(
                n_estimators=100,
                max_depth=20,
                min_samples_leaf=2,
                random_state=42,
                n_jobs=-1
            )
        )
    ]
)

rf_model.fit(X_rf_train, y_rf_train)

print("Random Forest Regressor trained successfully.")
print("Training records used:", len(X_rf_train))
```

```
Random Forest Regressor trained successfully.
Training records used: 100000
```

```
rf_predictions = rf_model.predict(X_test)

rf_mae = mean_absolute_error(
    y_test,
    rf_predictions
)

rf_mse = mean_squared_error(
    y_test,
    rf_predictions
)

rf_rmse = np.sqrt(rf_mse)

rf_r2 = r2_score(
    y_test,
    rf_predictions
)

print("RANDOM FOREST REGRESSION RESULTS")
print("--------------------------------")
print("MAE:", round(rf_mae, 2))
print("MSE:", round(rf_mse, 2))
print("RMSE:", round(rf_rmse, 2))
print("R2 Score:", round(rf_r2, 4))
```

```
RANDOM FOREST REGRESSION RESULTS
--------------------------------
MAE: 372.66
MSE: 215031.86
RMSE: 463.72
R2 Score: 0.989
```

```
# Use the same evaluation records for all models
evaluation_size = min(20000, len(X_test))

X_eval = X_test.sample(
    n=evaluation_size,
    random_state=42
)

y_eval = y_test.loc[X_eval.index]

# Training samples for computationally expensive models
X_train_sample = X_train.sample(
    n=min(100000, len(X_train)),
    random_state=42
)

y_train_sample = y_train.loc[X_train_sample.index]

print("Evaluation records:", len(X_eval))
print("Training sample records:", len(X_train_sample))
```

```
Evaluation records: 20000
Training sample records: 100000
```

```
# Store results from all eight models
model_results = []
trained_models = {}

def evaluate_model(model_name, model, X_fit, y_fit):
    print(f"Training {model_name}...")

    model.fit(X_fit, y_fit)

    predictions = model.predict(X_eval)

    mae = mean_absolute_error(y_eval, predictions)
    mse = mean_squared_error(y_eval, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_eval, predictions)

    model_results.append({
        "Model": model_name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2
    })

    trained_models[model_name] = model
```

```
    print("MAE:", round(mae, 2))
    print("MSE:", round(mse, 2))
    print("RMSE:", round(rmse, 2))
    print("R2 Score:", round(r2, 4))
    print("-" * 45)
```

```
ridge_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regressor", Ridge(alpha=1.0))
    ]
)

evaluate_model(
    "Ridge Regression",
    ridge_model,
    X_train,
    y_train
)
```

```
Training Ridge Regression...
MAE: 250.01
MSE: 83336.64
RMSE: 288.68
R2 Score: 0.9957
---------------------------------------------
```

```
lasso_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regressor", Lasso(alpha=0.01, max_iter=10))
    ]
)

evaluate_model(
    "Lasso Regression",
    lasso_model,
    X_train,
    y_train
)
```

```
Training Lasso Regression...
MAE: 250.0
MSE: 83303.45
RMSE: 288.62
R2 Score: 0.9957
---------------------------------------------
```

```
decision_tree_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "regressor",
            DecisionTreeRegressor(
                max_depth=10,
                random_state=42
            )
        )
    ]
)

evaluate_model(
    "Decision Tree Regression",
    decision_tree_model,
    X_train_sample,
    y_train_sample
)
```

```
Training Decision Tree Regression...
MAE: 805.43
MSE: 1021255.82
RMSE: 1010.57
R2 Score: 0.9479
----------------------------------------------
```

```
gradient_boosting_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "regressor",
            GradientBoostingRegressor(
                n_estimators=100,
                learning_rate=0.1,
                max_depth=3,
                random_state=42
            )
        )
    ]
)

evaluate_model(
    "Gradient Boosting Regression",
    gradient_boosting_model,
    X_train_sample,
    y_train_sample
)
```

```
Training Gradient Boosting Regression...
MAE: 401.66
```

```
MSE: 250089.05
RMSE: 500.09
R2 Score: 0.9872
----------------------------------------------
```

```
svr_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "regressor",
            SVR(
                kernel="rbf",
                C=100,
                epsilon=0.1
            )
        )
    ]
)

svr_sample_size = min(10000, len(X_train))

X_svr_train = X_train.sample(
    n=svr_sample_size,
    random_state=42
)

y_svr_train = y_train.loc[X_svr_train.index]

evaluate_model(
    "Support Vector Regression",
    svr_model,
    X_svr_train,
    y_svr_train
)
```

```
Training Support Vector Regression...
MAE: 314.95
MSE: 154616.06
RMSE: 393.21
R2 Score: 0.9921
----------------------------------------------
```

```
# Evaluate existing Linear Regression on the common test sample
linear_eval_predictions = linear_model.predict(X_eval)

linear_mae_eval = mean_absolute_error(
    y_eval, linear_eval_predictions
)

linear_mse_eval = mean_squared_error(
    y_eval, linear_eval_predictions
```

```
)

linear_rmse_eval = np.sqrt(linear_mse_eval)

linear_r2_eval = r2_score(
    y_eval, linear_eval_predictions
)

model_results.append({
    "Model": "Linear Regression",
    "MAE": linear_mae_eval,
    "MSE": linear_mse_eval,
    "RMSE": linear_rmse_eval,
    "R2 Score": linear_r2_eval
})


# Evaluate existing Random Forest on the common test sample
rf_eval_predictions = rf_model.predict(X_eval)

rf_mae_eval = mean_absolute_error(
    y_eval, rf_eval_predictions
)

rf_mse_eval = mean_squared_error(
    y_eval, rf_eval_predictions
)

rf_rmse_eval = np.sqrt(rf_mse_eval)

rf_r2_eval = r2_score(
    y_eval, rf_eval_predictions
)

model_results.append({
    "Model": "Random Forest Regression",
    "MAE": rf_mae_eval,
    "MSE": rf_mse_eval,
    "RMSE": rf_rmse_eval,
    "R2 Score": rf_r2_eval
})


# Display the eight models in order of RMSE
comparison_df = pd.DataFrame(model_results)

comparison_df = comparison_df.sort_values(
    by="RMSE",
    ascending=True
).reset_index(drop=True)
```

```
print("COMPARISON OF EIGHT REGRESSION MODELS")
display(comparison_df.round(4))

# Identify the model with the lowest RMSE
best_model_name = comparison_df.iloc[0]["Model"]

print("\nBest model based on RMSE:", best_model_name)
```

```
COMPARISON OF EIGHT REGRESSION MODELS
```

	Model	MAE	MSE	RMSE	R2 Score
0	Lasso Regression	249.9995	8.330345e+04	288.6234	0.9957
1	Ridge Regression	250.0134	8.333664e+04	288.6809	0.9957
2	Linear Regression	250.3421	8.366765e+04	289.2536	0.9957
4	Random Forest Regression	371.6150	2.134620e+05	462.0194	0.9891
5	Gradient Boosting Regression	401.6552	2.500891e+05	500.0890	0.9872
6	Decision Tree Regression	805.4298	1.021256e+06	1010.5720	0.9479